

Python Inbuilt Functions & Lambda Guide

Comprehensive reference sheet featuring 35+ core built-in functions, interactive examples, and lambda expressions.

1. MATHEMATICAL & NUMERIC FUNCTIONS

Function	Description	Code Example & Output
<code>abs(x)</code>	Returns the absolute value of a number.	<code>abs(-15) → 15</code>
<code>pow(base, exp)</code>	Returns base to the power exp (base ^{exp}).	<code>pow(2, 3) → 8</code>
<code>round(x, n)</code>	Rounds float x to n decimal digits.	<code>round(3.14159, 2) → 3.14</code>
<code>max(arg1, arg2...)</code>	Returns the largest item in an iterable or arguments.	<code>max(4, 12, 7) → 12</code>
<code>min(arg1, arg2...)</code>	Returns the smallest item in an iterable or arguments.	<code>min(4, 12, 7) → 4</code>
<code>sum(iterable)</code>	Sums all elements in an numerical iterable.	<code>sum([1, 2, 3, 4]) → 10</code>
<code>divmod(a, b)</code>	Returns tuple (a // b, a % b).	<code>divmod(10, 3) → (3, 1)</code>

2. TYPE CONVERSION & CREATION

Function	Description	Code Example & Output
<code>int(x)</code>	Converts x to an integer value.	<code>int("42") → 42</code>
<code>float(x)</code>	Converts x to a floating-point number.	<code>float("3.14") → 3.14</code>
<code>str(obj)</code>	Converts object to its string representation.	<code>str(100) → '100'</code>
<code>bool(x)</code>	Evaluates truth value of x.	<code>bool(0) → False</code>
<code>list(iterable)</code>	Converts an iterable to a list.	<code>list("abc") → ['a', 'b', 'c']</code>
<code>tuple(iterable)</code>	Converts an iterable to an immutable tuple.	<code>tuple([1, 2, 3]) → (1, 2, 3)</code>
<code>set(iterable)</code>	Converts an iterable to a set of unique elements.	<code>set([1, 2, 2, 3]) → {1, 2, 3}</code>
<code>dict(mapping)</code>	Creates a dictionary object.	<code>dict(a=1, b=2) → {'a': 1, 'b': 2}</code>
<code>chr(i)</code>	Returns string representing character at code i.	<code>chr(65) → 'A'</code>
<code>ord(c)</code>	Returns Unicode point value of character c.	<code>ord('A') → 65</code>

3. ITERABLES & SEQUENCE OPERATIONS

Function	Description	Code Example & Output
<code>len(s)</code>	Returns length (item count) of object.	<code>len([10, 20, 30]) → 3</code>
<code>range(start, stop)</code>	Generates an immutable sequence of numbers.	<code>list(range(1, 5)) → [1, 2, 3, 4]</code>
<code>sorted(iterable)</code>	Returns a new sorted list from iterable.	<code>sorted([3, 1, 2]) → [1, 2, 3]</code>
<code>reversed(seq)</code>	Returns a reverse iterator over sequence.	<code>list(reversed([1, 2])) → [2, 1]</code>
<code>enumerate(iterable)</code>	Adds a counter to iterable and returns tuple pairs.	<code>list(enumerate(['a'])) → [(0, 'a')]</code>
<code>zip(*iterables)</code>	Aggregates elements from each of the iterables.	<code>list(zip([1], ['a'])) → [(1, 'a')]</code>
<code>any(iterable)</code>	Returns True if any element in iterable is True.	<code>any([False, True]) → True</code>
<code>all(iterable)</code>	Returns True if all elements in iterable are True.	<code>all([True, False]) → False</code>
<code>map(func, iter)</code>	Applies function to all items in an iterable.	<code>list(map(str, [1, 2])) → ['1', '2']</code>
<code>filter(func, iter)</code>	Filters items based on truth value of function.	<code>list(filter(bool, [0, 1])) → [1]</code>

4. INPUT/OUTPUT & INSPECTION FUNCTIONS

Function	Description	Code Example & Output
<code>print(*objects)</code>	Prints objects to standard text stream.	<code>print("Hi") → Hi</code>
<code>type(object)</code>	Returns the data type of an object.	<code>type(42) → <class 'int'></code>
<code>isinstance(obj, cls)</code>	Checks if object is an instance of class/tuple.	<code>isinstance(5, int) → True</code>
<code>id(object)</code>	Returns unique integer memory identifier.	<code>id(x) → 1407...</code>
<code>dir(object)</code>	Returns list of valid attributes for object.	<code>dir([]) → ['append' ...]</code>
<code>help(object)</code>	Invokes built-in interactive help system.	<code>help(len) → Docs</code>
<code>callable(obj)</code>	Returns True if object appears callable.	<code>callable(print) → True</code>

5. LAMBDA FUNCTIONS (ANONYMOUS FUNCTIONS)

Lambda Syntax & Usage

A lambda function is a small, inline anonymous function defined using the syntax: `lambda arguments: expression`. It can take any number of parameters but can only evaluate a single expression.

```
# Complete Python Script Demonstrating Inbuilt Functions & Lambda

# 1. Basic Lambda Function Definitions
square = lambda x: x ** 2
add = lambda a, b: a + b

print(f"Square of 5: {square(5)}")          # Output: 25
print(f"Add 3 + 4: {add(3, 4)}")            # Output: 7

# 2. Using Lambda with Higher-Order Functions (map, filter, sorted)
numbers = [1, 2, 3, 4, 5, 6]

# map() + lambda to double numbers
doubled = list(map(lambda x: x * 2, numbers))
print(f"Doubled: {doubled}")                # Output: [2, 4, 6, 8, 10, 12]

# filter() + lambda to select odd numbers
odds = list(filter(lambda x: x % 2 != 0, numbers))
print(f"Odds: {odds}")                     # Output: [1, 3, 5]

# sorted() + lambda key for custom sorting
pairs = [(1, 'one'), (3, 'three'), (2, 'two')]
sorted_pairs = sorted(pairs, key=lambda item: item[1])
print(f"Sorted by word: {sorted_pairs}")    # Output: [(1, 'one'), (3, 'three'), (2, 'two')]
```